

Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2023/2024

Infraestruturas Computacionais Distribuídas

Relatório: Projeto (1ª Parte)

Turma: 42D

Nome: João Ramos	Número: 50730
Nome: Miguel Alcobia	Número: 50746
Nome: Fábio Pestana	Número: 50756

Professor: Porfírio Ferreira

Data: 14 de Abril de 2024

Índice

Introdução 1
Diagrama da arquitetura 2
Protocolos 2
Descrição dos componentes 3
Descrição das estruturas para persistência dos dados 4
Descrição dos protocolos ao nível da aplicação 4
Demonstração da aplicação 5
Conclusão 8
Bibliografia 9

Lista de Figuras

Figura 1 - Diagrama da arquitetura no contexto do trabalho 2
Figura 2 - Ecrã de Login 5
Figura 3 - Ecrã de registo 6
Figura 4 - Ecrã de Visualização de um perfil 6
Figura 5 - Alteração no IP Address 6
Figura 6 - Aviso de entrada de um jogador 7
Figura 7 - Visão do Cliente com a indicação do endereço IP definido anteriormente 7
Figura 8 - Partida de Othello na visão dos dois jogadores 7

Introdução

Para este projeto, o objetivo é consolidar e aplicar, conhecimentos sobre infraestruturas computacionais distribuídas, mais concretamente a conceção de arquiteturas e protocolos, estudados em aula, envolvidos na comunicação através de uma rede de computadores.

Este projeto foi dividido em duas partes, sendo que este relatório aborda a primeira delas. Nesta fase, desenvolveremos uma infraestrutura computacional que possibilite dois jogadores registados jogarem Othello, diretamente na consola, dentro da mesma rede.

Diagrama da arquitetura

Primeiramente, optou-se por uma arquitetura Thin Client, porque dado o contexto do trabalho (um jogo de Othello multijogador em rede) esta arquitetura enquadraria-se melhor para a sua resolução.

Contudo, devido a dificuldades na sua implementação, optou-se pela arquitetura peer-to-peer, embora esta segunda arquitetura apresente mais desvantagens, que serão abordadas mais adiante.

O seguinte diagrama, representa a arquitetura inserida no contexto do trabalho:

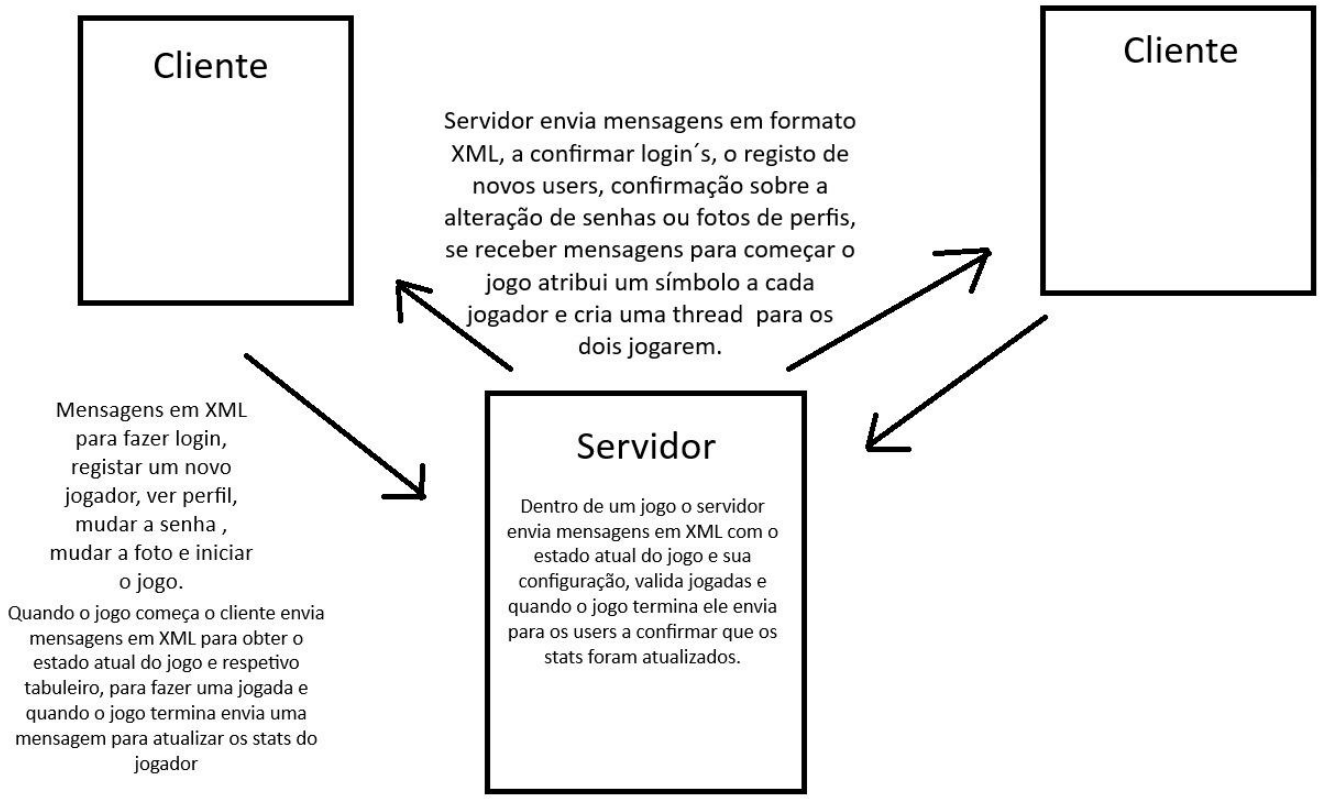


Figura 1 - Diagrama da arquitetura no contexto do trabalho

Protocolos

Os protocolos utilizados foram TCP e UDP nos portos 5025 e 5026, respetivamente. O servidor utilizado no trabalho tem por base um servidor TCP concorrente, enquanto o protocolo de descoberta da arquitetura utilizada (peer-to-peer) é baseado em UDP.

O TCP por ser um protocolo mais fiável é uma boa escolha para ser a base do servidor, pois garante a entrega, preserva a ordem de transmissão e não são aceites duplicados. Além de um processo poder trabalhar simultaneamente com várias conexões, o que é bastante útil dado o contexto do trabalho, onde temos dois jogadores a jogar Othello em rede.

O UDP no cliente embora não seja não fiável como o TCP por não garantir as entregas, por exemplo. Contudo, como o Othello é um jogo por turnos, os jogadores jogam à vez, a perda ocasional de pacotes tende a não impactar o jogo. O protocolo UDP também permite *broadcast* o que facilita a procura por jogadores, mas tem custos na segurança que é um ponto que será analisado mais à frente.

O TCP utiliza mecanismos de controlo de congestionamento que possibilitam a retransmissão de pacotes perdidos, aumentando o congestionamento da rede. O UDP, por outro lado, não possui esses mecanismos, o que significa que ele causa menos tráfego na rede.

Descrição dos componentes

Fundamentalmente, podemos dividir o trabalho em três grandes partes: Servidor, Cliente e Base de Dados.

O servidor é responsável pela gestão do jogo e pela interação dos dois jogadores. Os jogadores à medida que entram são colocados numa lista de espera (modo FIFO, "First in - First Out") e ao ser detetado um oponente, o servidor coloca os dois num jogo de Othello. Tanto o servidor como os clientes são implementadas com a ajuda de sockets.

A leitura e escrita de dados na socket é feita através de objetos `BufferReader` e `PrintWriter` respetivamente, onde cada cliente tem um par destes objetos associado a ele. Cada jogo é gerido por uma `Thread`.

Esta gestão é feita a partir do envio de strings XML, que depois são transformadas em Documentos XML. Aprofundaremos o funcionamento desta abordagem ao falar dos protocolos ao nível da aplicação mais à frente.

O cliente faz o envio e a leitura de mensagens tal como o servidor, mas na perspetiva do jogador. Ele envia as mensagens das jogadas, de autenticação e todos os métodos associados à classe User, que gere os seus componentes: foto, username e password que são os campos sujeitos a alteração pela mão do utilizador.

A base de dados é baseada em XML e validada por ficheiros XSD. Os Users e os países (que são utilizados na atribuição da nacionalidade) são organizados num ficheiro XML e cada um tem um XSD para a respetiva validação. O ficheiro XML dos utilizadores tem toda a informação dos utilizadores: username, password (codificada em SHA256), nacionalidade, foto de perfil (codificada em base64), o número de vitórias, número de derrotas, número de jogos realizados, o tempo total gasto em partidas e tempo médio por jogo de Othello.

Descrição das estruturas para persistência dos dados

Para garantir a persistência dos dados usou-se ficheiros XML e XSD. Os utilizadores são todos gravados num ficheiro XML e têm um ficheiro XSD para fazer a validação do mesmo. Desta forma, garantimos a consistência e a padronização em relação aos utilizadores.

As mensagens entre os clientes e o servidor também funcionam à base de XML e da respetiva validação com um ficheiro XSD. O cliente envia uma string no formato XML para o servidor e este consulta a informação recebida como um documento XML.

No próximo tópico o protocolo no qual estas estruturas foram utilizadas será explicado com mais detalhe.

Descrição dos protocolos ao nível da aplicação

O XML foi utilizado para o envio de mensagens entre o servidor e os clientes e também para criar e guardar as informações sobre os utilizadores.

Começando pela abordagem feita nos utilizadores, estes ao serem criados são gravados num ficheiro xml (`users.xml`), tendo um nó raiz `"users"`, depois cada user é representado por um nó `"user"`, que tem como filhos as várias informações que dizem respeito a esse user como username, password, nacionalidade, data de nascimento, número de vitórias, derrotas, número de jogos realizados, o tempo de jogo que o jogador já gastou em partidas e por último o tempo médio por jogo de Othello.

Esta informação depois é validada por um users.xsd que valida estes campos e as suas devidas restrições. Por exemplo, para o username, criou-se o tipo simples `UsernameType` que tem como restrição o tipo de caracteres aceites e um número máximo e mínimo de letras que o username tem de possuir.

As nacionalidades também têm um xml (`paises.xml`) e um xsd para validação (`paises.xsd`). O seu XML tem como nó raiz `"paises"` que tem como filhos os nós `"masculinas"` e `"femininas"`, ambos compostos por vários nós `"nacionalidade"`, separando assim o género das nacionalidades.

No que diz respeito às mensagens entre o servidor e o cliente, foi definido num ficheiro xsd (`othello-metodos-cli.xsd`) com o formato a ser respeitado para o envio de mensagens do servidor para o cliente no formato XML. O cliente pode enviar vários métodos diferentes para o servidor: fazer login (login), registar novo jogador (user), ver perfil (perfil), alterar a senha (changePass), alterar a sua foto (changePic), atualizar os stats do jogador após jogo (updateUser), iniciar jogo (iniciar), obter o tabuleiro (obter) e estado do jogo e realizar jogada (jogar).

O formato destas mensagens é:

- Abrir a *tag* `método` para declarar o envio de um método;
- Abrir a *tag* do método que se pretende enviar e os respetivos atributos.

Por exemplo, o método `iniciar` tem como atributos o username (`nickname`) do utilizador. Para enviar este método é construída a seguinte string XML:

```
<metodo><iniciar nickname=[username]/><metodo>
```

Quanto ao servidor, recorre a um socket para ler as mensagens enviadas pelo cliente. Transforma a string XML em um Document XML de facto e através do método `getMethod()`, ao qual se passa o Documet XML e o nome do método desejado, obtém-se o corpo da função desejada. Depois basta fazer *get* do atributo que se quer.

Por exemplo, o método `runIniciar()` (que lê a informação passada pelo método iniciar, descrito acima, enviado pelo cliente), recebe um socket e um símbolo, que é o símbolo associado ao jogador. Com a ajuda de um objeto `BufferReader` lê-se a informação do socket, que é a string XML.

Em seguida transforma-se essa string num Document XML e chama-se o `getMethod()` com os devidos argumentos e extrai-se o atributo `nickname`:

```
Document x = getNext(is);
String Nome = getMethod(x,"iniciar").getAttribute("nickname");
```

Este procedimento é usado para outros métodos, incluindo os que gerem a mecânica do jogo.

Demonstração da aplicação

Segue algumas *prints* de exemplo da aplicação.

```
*** Login ***
1 - Desejo Registar-me?
2 - Desejo Fazer login
2
<<< ***** Indique o seu nome de utilizador:
Teste5
<<< ***** Indique a sua senha:
12345
Login com sucesso

*** Menu User ***
1 - Ver Perfil
2 - Alterar senha
3 - Alterar foto
4 - Jogar Othello
```

Figura 2 - Ecrã de Login

```
*** Login ***
1 - Desejo Registar-me?
2 - Desejo Fazer login
1
Nome do utilizador: mikos
Senha para autenticação: pteste123
Ano de nascimento: 2004
Mês de nascimento: 01
Dia de nascimento: 21

Nacionalidade do utilizador: portuguesa
Indique o caminho para o ficheiro que contém a fotografia desejada tipo pass: player3.jpg
 Imagem:
  Path = src/main/webapp/player3.jpg
  Mime = jpg
Imagem: Conteudo em memória! (45594)
Novo User criado com sucesso!
```

Figura 3 - Ecrã de registo

```
*** Menu User ***
1 - Ver Perfil
2 - Alterar senha
3 - Alterar foto
4 - Jogar Othello
1
----------------------

Nome do Jogador: Teste5
Data de Nascimento: 2000-12-21
Nacionalidade: portuguesa
Idade: 23
Número de vitórias: 5
Número de derrotas: 2
Numero de jogos: 7
Tempo total gasto em jogos: 4 minutos, 49 segundos, 407 milissegundos.
Tempo médio por jogo jogos: 41 segundos, 343 milissegundos.

----------------------
```

Figura 4 - Ecrã de Visualização de um perfil

Como último colocou-se dois computadores para jogar em rede. O IP Address da Classe de um dos computadores para o IP do próprio PC, como mostrado abaixo:

```
   IPv4 Address. . . . . . . . . . . : 192.168.1.81
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : fe80::ce19:a8ff:fe96:d7bf%3
                                       192.168.1.254

Ethernet adapter Ligação de Rede Bluetooth:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

C:\Users\fabio>
```

```
public class Jogador {

    // **Constantes:**

    /**
     * Host padrão do servidor (endereço IP).
     */
    private final static String DEFAULT_HOST = "192.168.1.81"; //192.168.1.81

    /**
```

Figura 5 - Alteração no IP Address

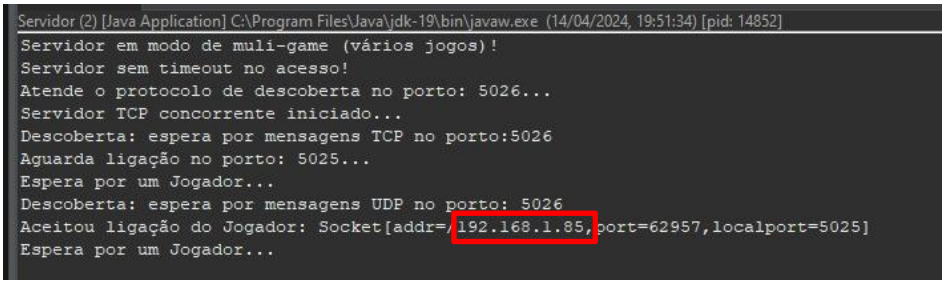


Figura 6 - Aviso de entrada de um jogador

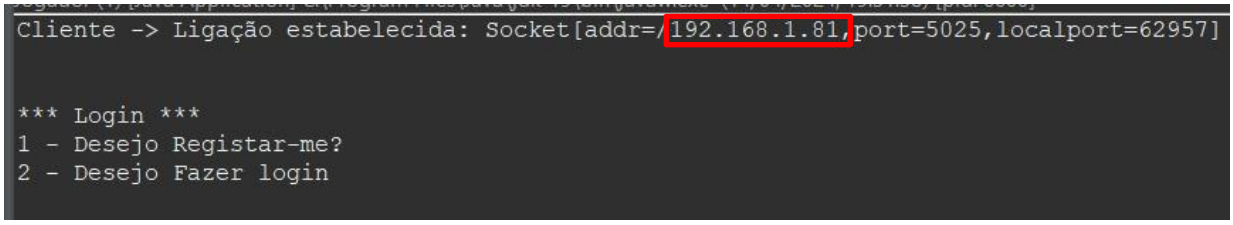


Figura 7 - Visão do Cliente com a indicação do endereço IP definido anteriormente



Figura 8 - Partida de Othello na visão dos dois jogadores

Conclusão

O grupo, no começo, pretendia seguir uma arquitetura Thin Client, pois apresenta como principal vantagem não sobrecarregar o cliente com o processamento dos dados, visto que estes serão tratados pelo servidor.

Contudo, o grupo teve dificuldades na sua implementação e acabou por optar-se por uma arquitetura peer-to-peer que, apesar das suas desvantagens, é mais facilmente implementada. Algumas das desvantagens da arquitetura peer-to-peer são, por exemplo, problemas de sincronismo e de segurança.

Devido à inexistência de um nó central, que seja responsável pela gestão da rede, será mais complexa a descoberta dos endereços dos nós da rede.

Já um servidor Thin Client, pode implementar medidas de segurança para registar o comportamento dos jogadores de forma mais eficaz, impedindo assim a ocorrência de fraudes e furtos aos dados dos utilizadores.

Mesmo com esta alteração ao nível da arquitetura, foram alcançados os objetivos desejados de forma satisfatória.

Bibliografia

Consulta

Ferreira, Porfírio & Remédios Diogo. (2024) *Infraestruturas Computacionais Distribuídas - Comunicação entre Processos - Sockets.* PowerPoint via Moodle.

Nacionalidade: Brasileiro/Brasileira? - Ciberdúvidas da Língua Portuguesa. (n.d.). Ciberduvidas.iscte-Iul.pt. https://ciberduvidas.iscte-iul.pt/consultorio/perguntas/nacionalidade-brasileirobrasileira/5328

Código:

Classes disponibilizadas, via Moodle, pelo professor Porfírio Ferreira em diversos projetos WAR, foram de grande ajuda para a elaboração do trabalho.